

Traffic Detection Pipeline: Final Project Report

1. Dataset & Model Origins (References)

To achieve high generalization across diverse urban environments, the team utilized a multi-stage data acquisition and architectural integration strategy.

Datasets Used:

- Phase I Foundation:** A curated subset of the **BDD100K (Berkeley DeepDrive)** dataset focusing on four primary traffic classes.
- Phase II Integration:** To improve robustness, we merged the BDD100K subset with the **UA-DETRAC** dataset, which provides high-density traffic camera footage.
- Ingestion & Processing:** All data was managed via **Roboflow** to standardize annotation formats. We utilized a custom Python script, `merge_datasets.py`, to map disparate class IDs into a unified 10-class system and synchronize image/label pairs.

Model Origins:

- YOLOv8s:** Sourced from the official **Ultralytics** repository, utilizing the single-stage anchor-free `yolov8s.pt` architecture.
- SSDLite320 (MobileNet V3):** Sourced from the official **PyTorch Hub / Torchvision** library (`torchvision.models.detection`), optimized for edge device efficiency.

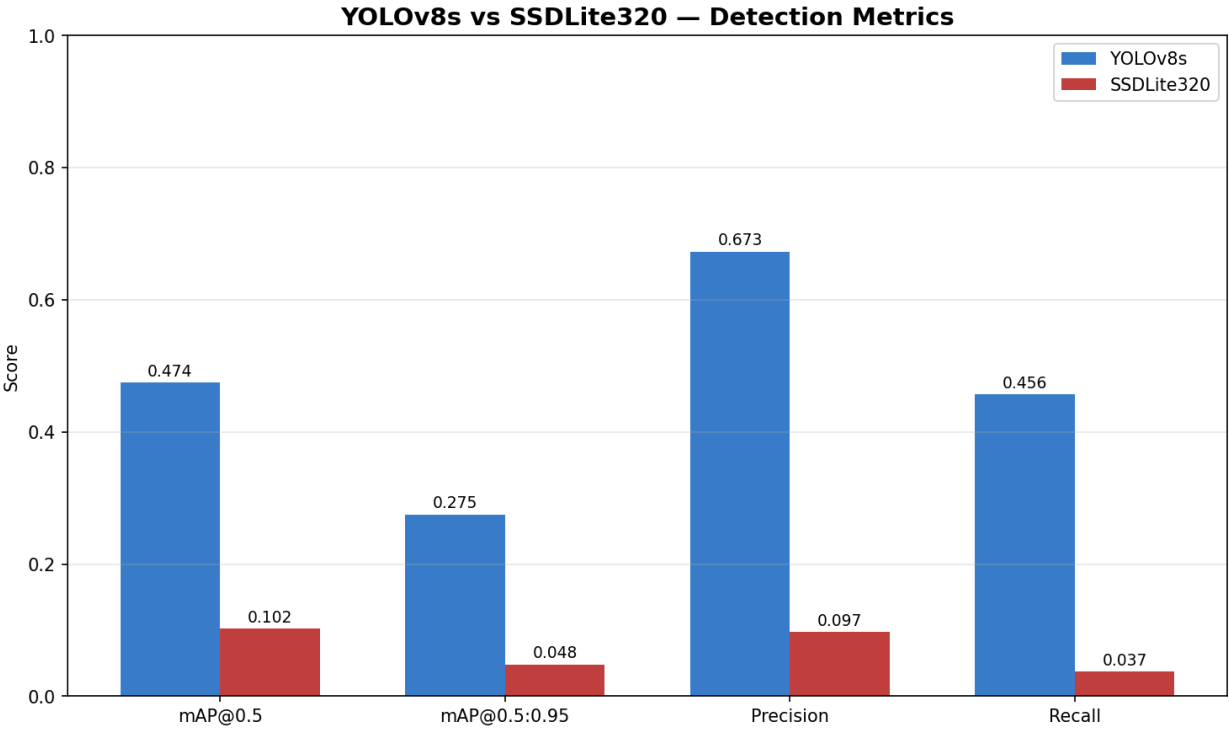
2. Model Evaluation & Comparison

The models were evaluated on a validation split of 1,661 images. Unlike the initial Phase I metrics, the fine-tuned YOLOv8s model demonstrated significant convergence and superiority over the SSDLite architecture.

Core Metrics Comparison

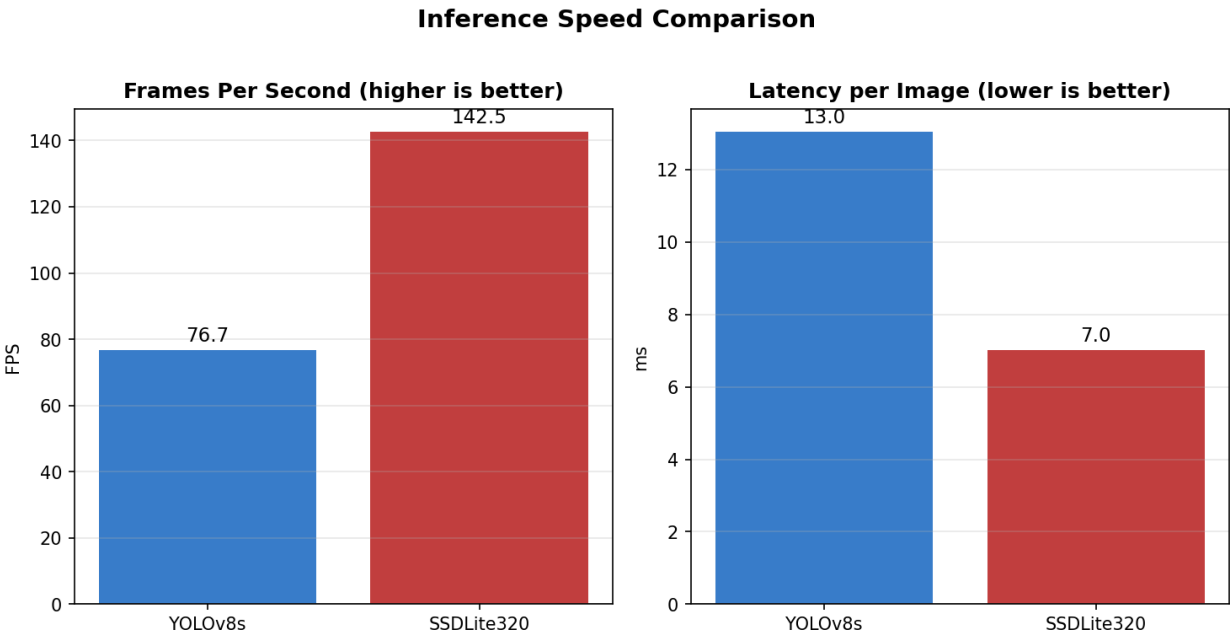
Metric	YOLOv8s	SSDLite320	Winner
mAP@0.5	0.4744	0.1025	YOLOv8s
mAP@0.5:0.95	0.2749	0.0483	YOLOv8s
Precision	0.6730	0.0970	YOLOv8s

Metric	YOLOv8s	SSDLite320	Winner
Recall	0.4357	0.0373	YOLOv8s



Inference Speed & Efficiency

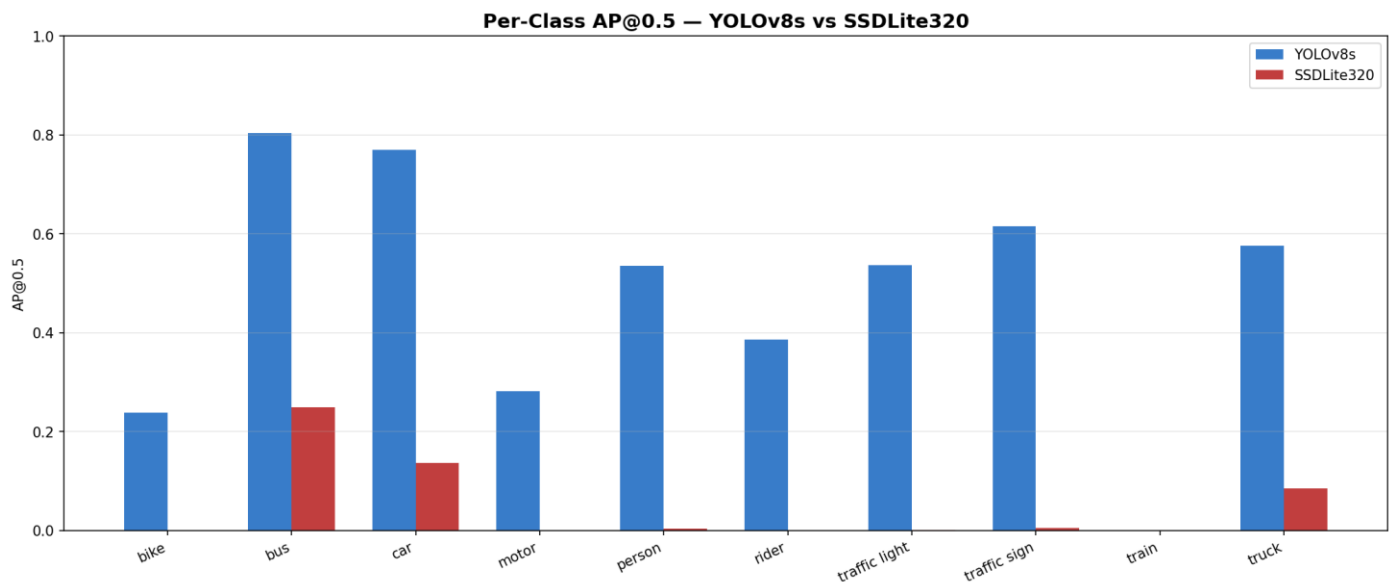
Metric	YOLOv8s	SSDLite320	Winner
FPS	76.75	142.51	SSDLite
Latency (ms)	13.03	7.02	SSDLite



Per-Class AP@0.5 Performance

YOLOv8s showed exceptional results in identifying major vehicle geometries compared to the underfitting observed in SSDLite.

- **Bus:** YOLOv8s achieved **0.8033** AP vs SSDLite's 0.2495.
- **Car:** YOLOv8s achieved **0.7705** AP vs SSDLite's 0.1368.
- **Truck:** YOLOv8s achieved **0.5764** AP vs SSDLite's 0.0851.
- **Traffic Sign:** YOLOv8s achieved **0.6153** AP vs SSDLite's 0.0056.

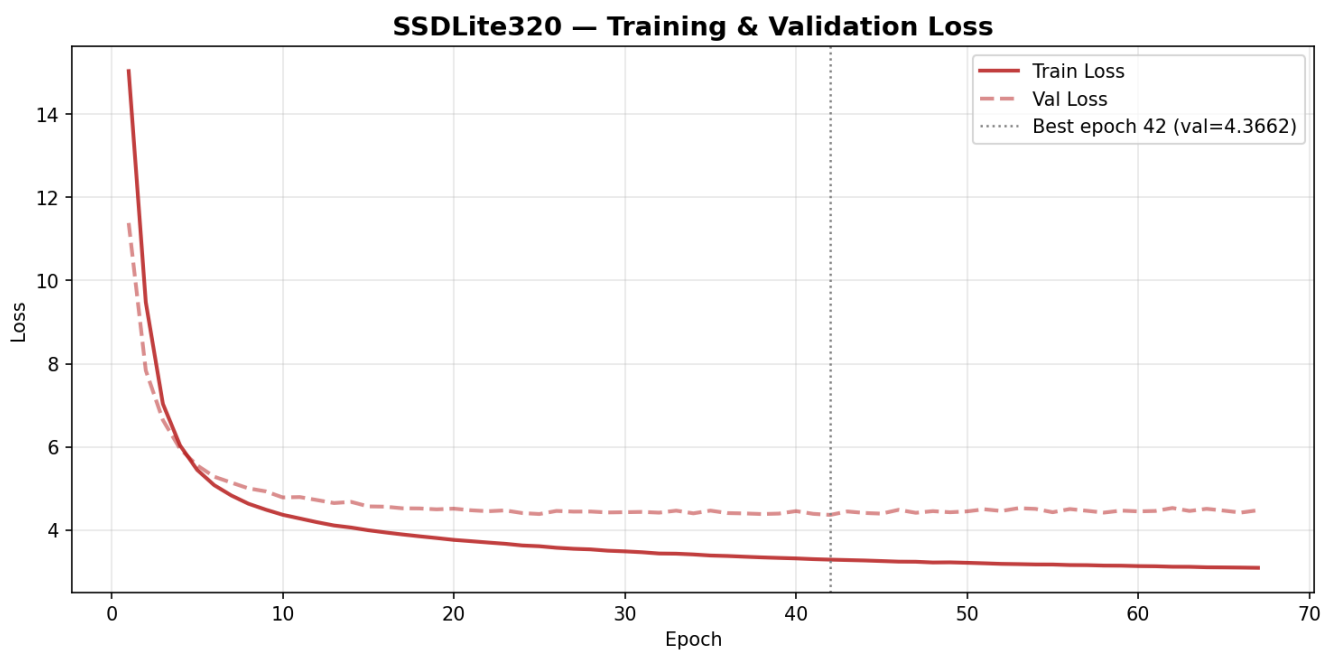
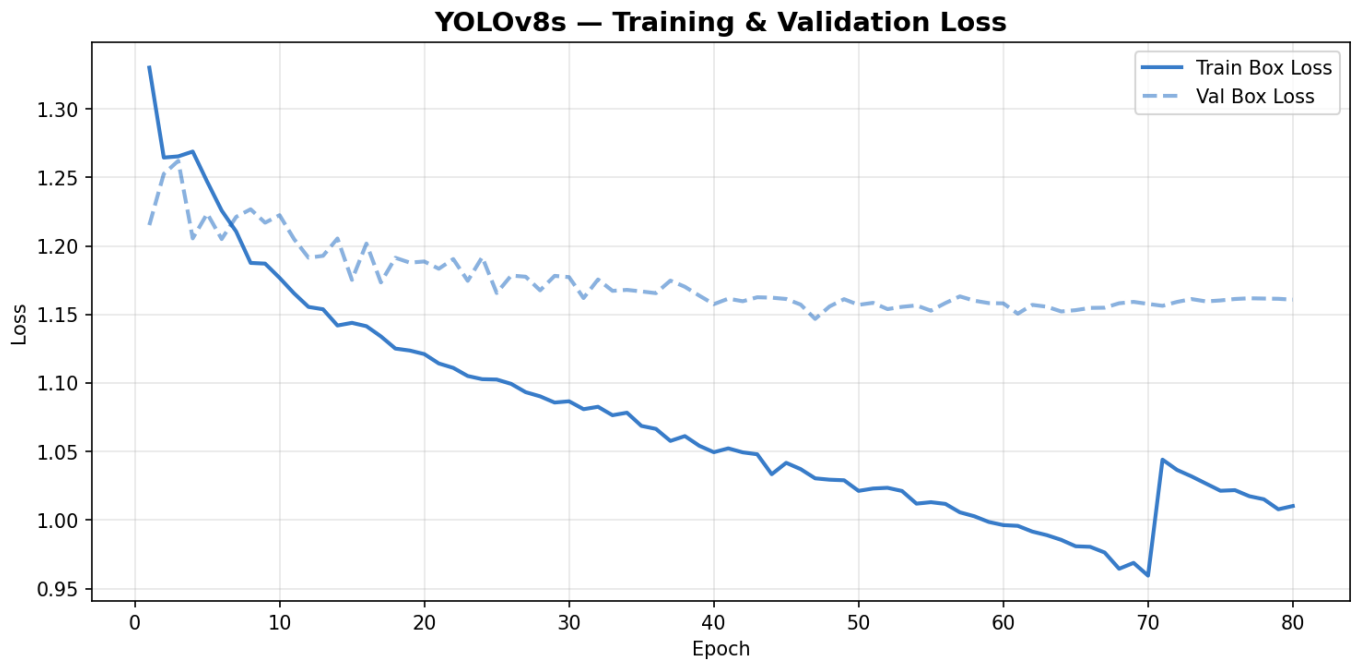


3. Team Contributions & Implementation

The project workload was distributed across three core engineering roles:

- **Data Engineering & Preprocessing:** One member focused on the Roboflow ingestion, the `merge_datasets.py` logic to resolve class ID conflicts between datasets, and the `convert_to_voc.py` script for cross-architecture label compatibility.
- **YOLO & SSD Training:** Another member managed the training execution for both models. This included configuring the `trainer.py` wrapper for YOLOv8s and the `ssd_trainer.py` script for the PyTorch implementation, managing hyperparameters, and generating training health plots.

- **Application & Deployment (Individual Contribution):** My primary responsibility was building the operational inference pipeline. This included the BaseDetector abstraction layer to allow model hotswapping, the custom IoU-based SimpleTracker for object persistence, and the stateful StatsCollector to log real-time vehicle counting and FPS data to JSON.



4. Final Conclusion

Based on the empirical evidence, **YOLOv8s is the vastly superior model for this dataset.** While SSDLite320 offers higher raw inference speeds (142 FPS), it fails to achieve acceptable detection accuracy for safety-critical traffic applications. YOLOv8s provides a balanced profile—achieving high detection rates across all major classes (buses, cars, trucks) while maintaining an inference speed of **76.75 FPS**, which far exceeds the 30 FPS requirement for real-time video processing. We recommend the final deployment utilize the YOLOv8s weights for all operational traffic monitoring tasks.